MODULE 6 · LEVEL 300

Deployment: Shipping Safely with AI

From a tested build to a safe release - pipelines, rollouts, and failure prediction, with GitHub Copilot

Day 3 · 90 minutes · Builds on Module 5 (the trusted suite) · Novnex

Where we are, and today's shape

approx 2 min

Module 5 gave us a suite we trust. Today we put it to work - shipping changes to users quickly and safely.

Framing

Why AI helps deployment, and the CI/CD picture.

~18 min

Deployment with Copilot

Pipelines, fixes, workflows, IaC, rollouts, prediction.

~51 min

Apply

By track, anti-patterns, the hand-off, and the lab.

~21 min

Deployment is where mistakes reach users fastest - so here, AI speeds you up only if it also keeps you safe.

Learning objectives

approx 2 min

By the end of this module you will be able to:

Build & fix pipelines

Author CI/CD with Copilot, and diagnose and fix failing runs fast.

Release safely

Design canary and progressive rollouts, and generate clean release notes.

Anticipate failure

Use AI to flag risky releases before they reach users.

Keep control

Automate the pipeline while humans approve what actually ships.

Why AI in deployment - speed with safety

approx 4 min

It removes the friction

- **Pipelines are fiddly:**
- YAML, permissions, caching - Copilot drafts them.
- **Failures are cryptic:**
- it explains a red build in plain language.
- **Toil disappears:**
- release notes, changelogs, boilerplate config.
- **Fewer stalls:**
- unblock a broken pipeline in minutes, not hours.

But this is the sharp end

- A bad deploy reaches every user immediately
- Pipelines hold secrets and production access
- An agent in a pipeline can do real damage
- So speed is only a win if safety comes with it

Where AI helps across deployment

approx 4 min

Author pipelines

Draft and refine GitHub Actions.

Fix failures

Explain and repair red builds.

Infrastructure

Generate and review IaC.

Release notes

From your merged PRs and commits.

Rollouts

Design canary and progressive releases.

Predict failure

Flag risky changes before deploy.

We'll take these in order. The rule for this phase: automate the steps, but keep a human on the ship button.

The pipeline - where Copilot plugs in

approx 5 min

Copilot helps at every stage - but 'Deploy' is where the guardrails and the human approval live.

PART 2

Deployment with Copilot

Pipelines, failing builds, workflows, infrastructure, release notes, rollouts and prediction.

approx 51 min

Author pipelines with Copilot

approx 5 min

Describe the pipeline you want in plain language; Copilot drafts the GitHub Actions YAML - you review it.

.github/workflows/ci.yml (drafted)

```
on: [pull_request]
jobs:
  test:
    steps:
      - run: pip install -r requirements
      - run: pytest -q   # gate on green
      - uses: github/codeql-action
```

Draft, then review

- **From intent:**
- 'test on PRs, cache deps, run scanning'.
- **Explains the YAML:**
- ask what each step does and why.
- **Least privilege:**
- check the permissions it grants.
- **Pin versions:**
- you review actions and their sources.

Fix failing pipelines fast

approx 5 min

Red build -> green

```
x  test job failed (exit 1)
ModuleNotFoundError: no module 'pytest'
> Explain error  ->  deps not installed
> Fix with Copilot
+ add: pip install -r requirements.txt
```

Unblock in minutes

- **Explain error:**
- plain-language cause on a failed run.
- **Fix with Copilot:**
- the agent proposes a fix and tags you.
- **Common culprits:**
- deps, env, permissions, flaky steps.
- **You approve:**
- review the fix before it merges.

Author & debug workflows with agents

approx 4 min

Agents for CI/CD

- **Build workflows:**
- describe the automation; the agent drafts it.
- **Debug them:**
- 'why does this job time out?' - and fix it.
- **Copilot in the pipeline:**
- the CLI can run inside a workflow step.
- **Repetitive ops:**
- scripted, reviewable, version-controlled.

Kept on a leash

- Workflow changes go through pull-request review
- Secrets stay in the vault, never in prompts
- Actions on protected environments need approval
- Automation you can read beats a magic script

Infrastructure as Code with Copilot

approx 4 min

Terraform (drafted & reviewed)

```
resource "aws_s3_bucket" "assets" {
  bucket = "acme-portal-assets"
}
# Copilot review: enable encryption +
# block public access (least privilege)
```

Code your infra

- **Draft it:**
- Terraform, Bicep, Kubernetes manifests.
- **Review it hard:**
- infra mistakes are expensive and public.
- **Security defaults:**
- ask it to check encryption and access.
- **Plan before apply:**
- AI drafts; you run the plan and approve.

Release notes & changelogs

approx 4 min

From merged PRs -> notes

```
## Release 2.4
- Self-service password reset (#128)
- Rate limiting on auth endpoints (#142)
- Fix: legacy accounts without email (#151)
```

Free the boring bit

- **From the PRs:**
- Copilot summarises what actually changed.
- **Audience-aware:**
- a technical changelog or a user-facing note.
- **Consistent:**
- same format, every release.
- **You edit tone:**
- a quick pass, not a blank page.

Smart rollout strategies

approx 5 min

Ship gradually

- **Canary:**
- release to a small slice; watch before going wide.
- **Progressive:**
- ramp traffic as health metrics stay good.
- **Feature flags:**
- decouple deploy from release; toggle safely.
- **Design with AI:**
- 'draft a canary plan with rollback triggers'.

Fail small, recover fast

- A blast radius of 1% beats a blast radius of 100%
- Define the rollback trigger before you deploy
- AI can draft the automation and the runbook
- The strategy is yours; AI removes the busywork

Predict failure before it ships

approx 4 min

Risky change signals

Large diffs, hot files, weak test coverage - flagged pre-merge.

Deployment risk score

A read on how likely this release is to cause trouble.

Extra gates when risky

Trigger more review or a slower rollout for high-risk changes.

Faster diagnosis

When something does break, AI speeds the root-cause hunt.

Deployment safety & approvals

approx 4 min

Agents can now act inside pipelines - which is powerful and dangerous. The guardrails are the whole game.

Environment approvals

Production deploys require a named human to approve.

Least privilege

Pipelines and agents get only the access they need.

Secrets stay secret

Vaulted, never in prompts, logs or code.

Rollback ready

A tested path back is part of every release.

An agent's pipeline change is reviewed like any other - and no agent gets the production ship button unattended.

Ground deployment in your platform

approx 4 min

A Space with your real pipelines, runbooks and standards makes Copilot's suggestions fit your platform.

Space: "Platform & Delivery"

```
repos:  acme/infra, acme/workflows
files:  runbooks/, deploy-standards.md
instructions:
  Use our Actions patterns and naming.
  Require approvals on prod; least priv.
```

Fits your platform

- **Your patterns:**
- pipelines follow your conventions.
- **Runbook-aware:**
- suggestions cite your procedures.
- **Safer defaults:**
- approvals and least privilege built in.
- **Consistent:**
- every team ships the same way.

Live demo: from red build to release

approx 6 min

Watch a pipeline recover

- A pull request turns the pipeline red
- 'Explain error' - the cause in plain language
- 'Fix with Copilot' - the agent proposes a fix
- We review and approve; the build goes green
- Copilot drafts the release notes from the PRs

Flow

- **1.**
- Open the failing run.
- **2.**
- Explain the error.
- **3.**
- Fix with Copilot.
- **4.**
- Review, approve, green.
- **5.**
- Generate release notes.

Responsible deployment with AI

approx 4 min

Auto-ship, unreviewed	Letting AI deploy to prod on its own.	**Guard:** A named human approves production; agents never ship unattended.
Secrets in prompts/logs	Leaking credentials through AI or output.	**Guard:** Vault secrets; content exclusion; scrub logs; least privilege.
No way back	Deploying with no rollback plan.	**Guard:** Every release has a tested rollback and a defined trigger.
Blind trust in a fix	Merging an AI pipeline fix unread.	**Guard:** Review the fix and the permissions it touches, like any change.

PART 3

Apply it

By track, the anti-patterns, the hand-off to maintenance, and your lab.

approx 21 min

Deployment with Copilot - by track

approx 4 min

Web / Drupal

GitHub Actions for build, test and deploy; environment approvals; rollback.

Data & Analytics

Orchestrate pipelines (jobs, dbt); gate on data tests; safe backfills.

Power BI

Deployment pipelines (dev/test/prod); dataset refresh and validation gates.

RPA

Promote bots across environments; versioning and controlled release.

Anti-patterns to avoid

approx 4 min

Don't

- Let an agent deploy to prod unattended
- Ship 100% on the first push
- Put secrets in prompts, logs or code
- Merge an AI pipeline fix without reading it
- Trust a generated pipeline you don't understand

Do

- Keep a human on the production ship button
- Deploy gradually - canary before full rollout
- Have a tested rollback for every release
- Review pipeline and IaC changes like code
- Ground Copilot in your platform standards

The hand-off to maintenance

approx 3 min

Shipping isn't the finish line - it's the start of operating. What you deploy, you now have to watch.

Observability in

Logs, metrics and traces wired up from day one.

Alerts that matter

Signal on the things that actually hurt users.

Runbooks ready

How to respond when the thing you shipped misbehaves.

Rollback proven

You've tested the way back, not just the way out.

Module 7 - Maintenance - is where AI helps you operate, triage and sustain what you've shipped.

Your lab: ship it safely

approx 2 min

Lab 6 (this afternoon) - on your own track

- Diagnose and fix a failing pipeline with your BYOK model in VS Code
- Draft or improve a CI workflow that gates on your tests from Lab 5
- Design a canary/rollout plan with a rollback trigger
- Generate release notes from a set of merged PRs

Pick a track; pair to review. The goal is a pipeline that ships safely - with a human approval and a way back.

Discussion & reflection

approx 4 min

Talk to the person next to you - three minutes:

- Where in your release process would AI help most - and where must a human stay in control?
- What's your rollback story today - and how would you make failure 'small'?

Pipeline toil	**Failing builds**	**Rollout safety**	**Secrets**	**Rollback**

We'll take a few answers before wrapping up.

Key takeaways

- AI removes deployment friction - pipelines, fixes, notes, config.
- Fix red builds fast: Explain error, then Fix with Copilot.
- Ship gradually - canary and progressive rollouts, with a rollback.
- Predict risk before deploy; add gates when a change looks risky.
- Automate the steps - but a human approves what ships, always.

What's next

Module 7

Maintenance & Support

then

Lab 6 this afternoon

Questions?